



Práctica 12

Objetivo

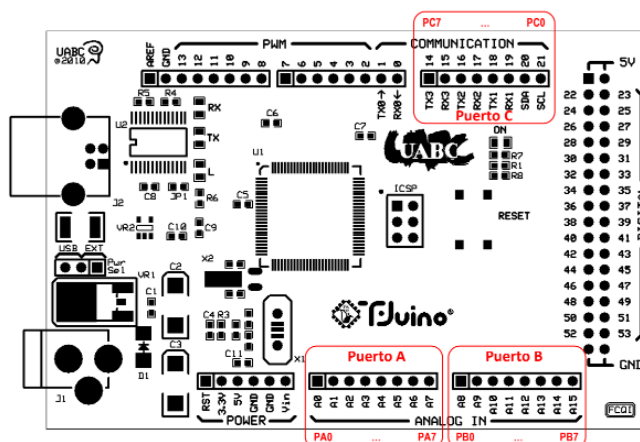
Seleccionar las instrucciones correctas en aplicaciones de sistemas basados en microprocesador mediante la distinción de su funcionamiento, de forma lógica y responsable.

Desarrollo Puertos

Ejemplo:

Asignación de puertos del PII-8255 en T-Juino

```
1 .model tiny
2 locals
3 .data
4 PA DW 0040h
5 PB DW 0041h
6 PC DW 0042h
7 RCtrl DW 0043h
8 PTOs_all_out DB 80h ;80h para indicar que todos los puertos son de salida
9
10 .code
11 org 100h
12
13 principal proc
14 .rep:
15     mov sp, 0fffh
16
17     mov dx, RCtrl ;en dx recibe el puerto
18     mov al, PTOs_all_out ;en al el dato a sacar
19     call outportb
20
21
22     mov dx, PA ;en dx recibe el puerto
23     mov al, 55h ;en al el dato a sacar
24     call outportb
25
26     jmp .rep
27
28     ret
29 endp
30
31 outportb proc
32     ;en dx recibe el puerto
33     ;en al el dato a sacar
34     out dx, al
35
36     ret
37 endp
38 end principal
```



1. Realizar lo siguiente:

- a) Copiar el código del ejemplo de puertos en el archivo P12_Prueba.asm
- b) Ensamblar y enlazar el archivo para obtener el P12_Prueba.com
- c) Cargar el archivo P12_Prueba.com a la placa T-Juino
- d) Ir trazando la ruta del código ensamblador con 't', para ir revisando como va actuando en los registros y los puertos el código de ejemplo.
- e) Al mismo tiempo poner leds en la salida del puerto para ver como saca el valor indicado por el código P12_Prueba.asm

2. Cree un programa llamado **P12.asm** que contenga las siguientes rutinas para el intérprete 80x86:

- a) **SetBit**: Activa un bit del registro AX. El número de bit a activar está dado por CL.
- b) **ClearBit**: desactiva un bit del registro AX. El número de bit a desactivar está dado por CL.
- c) **NotBit**: Invierte un bit del registro AX. El número de bit a invertir está dado por CL.

Ejemplo de uso del código:

```
mov ax, 4h
mov cl, 3
call SetBit
call printBin -> imprime en pantalla un 0000 1100
```

```
mov cl, 2
call ClearBit
call printBin -> imprime en pantalla un 0000 1000
```

```
mov cl, 0
call NotBit
call printBin -> imprime en pantalla un 0000 1001
```

3. Realizar las siguientes subrutinas en ensamblador para la placa T-Juino.

- **SetBitPort**: Manipula la información de un puerto dado por DX para activar un determinado bit. Es decir, mediante ella se puede activar (hacer uno) un bit del puerto. El número del bit está en el rango de 0 a 7 siendo el bit 7 es más significativo y está dado por CL.
- **ClearBitPort**: Manipula la información de un puerto dado por DX para desactivar un determinado bit. Es decir, mediante ella se puede desactivar (hacer cero) un bit del puerto. El número del bit está en el rango de 0 a 7 siendo el bit 7 es más significativo y está dado por CL.
- **NotBitPort**: Manipula la información de un puerto dado por DX para invertir un determinado bit. Es decir, mediante ella se puede invertir un bit del puerto. El número del bit está en el rango de 0 a 7 siendo el bit 7 es más significativo y está dado por CL.

Las 3 subrutinas creadas anteriormente para ser aceptadas como correctas se deben colocar 8 leds en la salida del puerto C y agregar aparte del código y las capturas de las pruebas e impresiones, se debe tener una fotografía de los leds encendidos en el orden correcto de acuerdo a cada prueba.

Ejemplo de uso del código:

```
mov dx, 42h
in al, dx
call printBin -> imprime en pantalla 0100 1100
```

```
mov dx, 42
mov cl, 4
call SetBitPort -> activa el bit 4 del puerto 42h del Tjuino siendo este 0101 1100
```

```
mov dx, 42
mov cl, 3
call ClearBitPort -> desactiva el bit 3 del puerto 42h del Tjuino siendo este 0101 0100
```

```
mov dx, 42
mov cl, 7
call NotBitPort -> invierte el bit 7 del puerto 42h del Tjuino siendo este 1101 0100
```

Se debe entregar un reporte con todo lo que le corresponde.
Deben utilizar cadenas de ayuda al usuario, ingresar el valor, etc.

Conclusiones y comentarios

Dificultades en el desarrollo

Referencias